

Relatório CodeReview

Autor da CodeReview: Anna Santoro

Desenvolvedor do código: Ana Carolina Correa

API Testada: [Restful-Booker](#)

Link do repositório e branch avaliada: [Repositório Compass - Main Branch](#)

Análise do CodeReview: [🔗](#)

Pontos Positivos [🔗](#)

- Separação clara entre testes e funcionalidades
 - Boa cobertura dos endpoints da API
 - Uso de dados aleatórios
 - Sessões HTTP configuradas corretamente
 - Validação de status code centralizada
-

Pontos de Melhoria [🔗](#)

- Falta de validação do conteúdo das respostas (payload)

Na maioria dos testes, só o código de resposta é verificado. Isso pode deixar passar erros mais sutis, como campos incorretos ou dados incompletos.

- Comparação de status code usando strings

A comparação entre `response.status_code` (número) e `"statuscode"` (string) pode causar falhas inesperadas. É melhor garantir que ambos estejam no mesmo formato (inteiro).

- Uso de variáveis globais

O uso de `randomfirstname` e `token` como variáveis globais pode causar problemas se os testes forem executados em paralelo ou em diferentes contextos.

Sugestões para Evolução [🔗](#)

- **Validar campos específicos nas respostas**

- Criar uma nova keyword como:

```
1 Validar campo na resposta
2     [Arguments] ${chave} ${valor_esperado}
3     ${valor}      Get From Dictionary ${response.json()} ${chave}
4     Should Be Equal ${valor} ${valor_esperado}
```

- **Adicionar testes de verificação após atualização ou exclusão**
 - **Criar um arquivo de variáveis:** Mover valores como a URL base, credenciais e datas fixas para um arquivo `variables.robot`.
 - **Implementar testes negativos:** Testar com dados inválidos ou cenários de erro (ex: login errado, reserva inexistente) para verificar o comportamento da API em situações inesperadas.
 - **Utilizar setup e teardown com `Suíte Setup`:** Para inicializar a sessão da API uma única vez no início da execução da suíte, economizando tempo.
-

Resumo [↗](#)

Este projeto de automação está **bem estruturado**, e já demonstra boas práticas importantes como reutilização de código, modularização e clareza. Como próximos passos, o foco pode ser: tornar os testes mais completos com validação de conteúdo, refatorar o uso de dados estáticos e explorar testes negativos para aumentar a robustez do conjunto de testes.